SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY
COURSE CODE: CSA5117

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Assessment Tool Weightage**:** 10%

QUESTIONS: **Total Marks: 50**

Annexure A

Debugging & Optimization Task

Code of Conduct:

I, Dharshavardhan (Reg. No. 192420039), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A
Debugging & Optimisation Task

1. Debugging & Optimization of Symmetric Encryption (DES / 3DES Implementation)
A Python program implementing **DES and 3DES encryption** produces inconsistent ciphertext for identical inputs across multiple runs.

A. Identify and fix logical and implementation errors in key scheduling and permutation steps.
B. Optimize the code to reduce execution time for large input datasets (≥10 MB).
C. Justify whether replacing DES/3DES with a modern cipher improves both security and performance.

2. Performance Analysis of Block Cipher Modes of Operation
A Python script implements **AES-like block cipher modes (ECB, CBC, CFB, OFB)** but suffers from high latency and memory inefficiency.

A. Debug incorrect padding and IV handling causing decryption failures.
B. Refactor the code to minimize redundant computations and memory usage.
C. Compare and optimize the implementation to determine the most efficient mode for real-time secure communication.

3. Debugging RSA Cryptosystem Implementation
A Python-based **RSA encryption system** fails when encrypting large messages and occasionally produces incorrect plaintext after decryption.

A. Identify issues in prime generation, key generation, and modular exponentiation.
B. Optimize the algorithm using efficient techniques (e.g., fast exponentiation, Chinese Remainder Theorem).
C. Evaluate the trade-offs between key size, security, and computational performance.

4. Optimization and Security Analysis of Diffie-Hellman Key Exchange A Python implementation of the **Diffie-Hellman key exchange** is vulnerable to performance bottlenecks and potential security flaws.

A. Debug incorrect shared key generation due to improper modular arithmetic.
B. Optimize exponentiation operations for large prime values.
C. Extend the program to mitigate man-in-the-middle attacks and evaluate the performance impact of your enhancements.

5. Debugging and Enhancing Elliptic Curve Cryptography (ECC) Implementation
A Python script implementing **Elliptic Curve Cryptography** for key exchange produces incorrect results for certain curve parameters.

A. Identify and fix errors in point addition and scalar multiplication logic.
B. Optimize the implementation using efficient algorithms (e.g., double-and-add).

C. Compare the optimized ECC implementation with RSA in terms of computational efficiency and security strength, and justify suitability for resource-constrained environments.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

5. Debugging and Enhancing Elliptic Curve Cryptography (ECC) Implementation

A. Identify and Fix Errors in Point Addition and Scalar Multiplication

ECC operates on points on an elliptic curve over a finite field. Incorrect results commonly occur when the code does not handle the point at infinity, uses incorrect modular arithmetic, or calculates the slope incorrectly. For distinct points P=(x■,y■) and Q=(x■,y■), the slope is λ = (y■−y■)(x■−x■)■¹ mod p. For point doubling, λ = (3x■²+a)(2y■)■¹ mod p. The inverse must be a modular inverse, not ordinary division. If x■=x■ and y■=−y■ mod p, the result is the point at infinity. Point coordinates must be reduced modulo p after each operation.

Example corrected point-addition logic (prime field)

```
def point_add(P, Q, a, p):
    # None represents the point at infinity
    if P is None:
        return Q
    if Q is None:
        return P

    x1, y1 = P
    x2, y2 = Q

    # P + (-P) = point at infinity
    if x1 == x2 and (y1 + y2) % p == 0:
        return None

    if P == Q:  # Point doubling
        denominator = (2 * y1) % p
        if denominator == 0:
            return None
        slope = ((3 * x1 * x1 + a) *
                 pow(denominator, -1, p)) % p
    else:      # Addition of distinct points
        denominator = (x2 - x1) % p
        slope = ((y2 - y1) *
                 pow(denominator, -1, p)) % p

    x3 = (slope * slope - x1 - x2) % p
    y3 = (slope * (x1 - x3) - y1) % p
    return (x3, y3)
```

B. Optimize Scalar Multiplication Using Double-and-Add

Scalar multiplication computes kP by adding a point repeatedly. The inefficient approach performs roughly k additions. The double-and-add algorithm processes the binary digits of k and needs only O(log k) point operations, making it much faster for large scalars.

```
def scalar_multiply(k, P, a, p):
    result = None
    addend = P

    while k > 0:
        if k & 1:
            result = point_add(result, addend, a, p)
        addend = point_add(addend, addend, a, p)
        k >>= 1

    return result
```

The implementation should also validate that points lie on the selected curve, reject invalid parameters, and use a trusted cryptographic library for real applications. Basic double-and-add can reveal information through timing or power use; production code should use constant-time, side-channel-resistant routines.

C. ECC vs RSA: Efficiency and Security

Feature	ECC	RSA
Key size	Smaller keys for comparable security	Larger keys required
Memory / bandwidth	Lower storage and transmission needs	Higher storage and bandwidth needs
IoT efficiency	Often lower power and memory use	Can be more demanding, especially for private-key operations
Typical use	ECDH key exchange; ECDSA signatures	RSA key transport in legacy systems; RSA signatures

ECC is generally the better choice for resource-constrained environments because it offers comparable security with smaller keys and usually reduces memory, bandwidth, and energy requirements. The exact speed depends on the device, curve, library, and operation; RSA may be fast for public-key operations with a small exponent. For new systems, use modern parameters and established libraries rather than custom cryptographic code.

Conclusion

Correct point addition requires proper modular inverses and handling of the point at infinity. Double-and-add improves scalar multiplication from O(k) to O(log k) point operations. ECC is typically more suitable than RSA for low-power IoT devices due to its smaller keys and lower resource requirements.